

Name: **Amit Kshirsagar**

ID No: **2018AAPS0381G**

Floating Point Adder - Final Submission

Modules Build:

1. **Small_alu** - This module takes the exponents of both the given numbers as input and calculates the difference between the exponents and stores it. It corresponds to the Small Alu present in the Floating-point adder.
2. **Mux_greater_exponent** - This module also takes the exponents of both the given numbers as input, and it also receives a control line that indicates which exponent is greater than the other. If both the exponents are equal, then the exponent of number 1 is chosen by default (it does not matter which one is chosen as both have the same value). The control line it receives is the sign bit of the exponent difference. The exponent difference is calculated as the exponent of num1 – exponent of num2, so if num1's exponent is greater than the sign bit is 0 and exponent of num1 is chosen, else the exponent of num2 is chosen. This module corresponds to the 8-bit mux in the Floating-point adder.
3. **Mux_smaller_fraction** - This module takes in the fractional parts of both the numbers and two control lines and outputs the fractional part of the smaller number. In case the exponent of both the numbers are the same, then the fractional parts are compared directly. 1st control line(exp_diff_sign) is the sign bit of exponent difference, while the 2nd control line(checkbit) tells whether the exponent difference is 0 or not. If the difference is 0, then fractional parts are compared, and the smaller one is given as the output. If the exponent difference is non zero then comparing the exponent difference sign, the output is decided. This module corresponds to the mux present before shift right block in the floating-point adder.
4. **Mux_greater_fraction** - This module also receives the same input as mux_smaller_fraction, but in this case, it gives the fractional part corresponding to the greater number. This also outputs a great_line, which tells which number is greater. This module corresponds to the mux present before Big ALU in the floating-point adder.
5. **Absofdiff** - This module is given the exponent difference as the input, and the output is the absolute value of the exponent difference.

6. **Shift_right** - This module takes the absolute value of the exponent difference (shift value) and the smaller number's fractional part. It right shifts the fractional part (has 23bits +1 bit of normalized form) by the shift value (exponent difference) and gives the fractional part output. This module corresponds to the shift right block in the floating-point adder.
7. **Big_ALU** - this module takes the following inputs- fractional part of the greater number, the right-shifted fractional part, the sign bits of both the input numbers and a control line. The sign bits tells whether to perform addition or subtraction operation, and the control line tells which number is greater in case the exponent part is the same. The fractional parts have 23 bits, but 1 extra bit has been added to it that corresponds to the 1 bit present in the normalized form before the decimal. Thus the Big ALU performs the addition/subtraction of two 24 bit numbers. Hence the output is 25 bits.
8. **Mux_normalize_fraction** - This module takes the fractional part output of Big ALU, fractional part output of rounding hardware and a control line and gives the output to the normalizing unit.
9. **Mux_normalize_exponent** - This module takes the greater exponent, output exponent part of rounding hardware, and control line and gives the output to the normalizing unit.
10. **Fract_shift_left_right** - This module takes the exponent part, and fractional part as input, these inputs either come from ALU operation for normalizing or come to this module after rounding has been carried out. This module normalizes the 25 bits output from the ALU operation and outputs a fractional part that has valid 23 bits and 1 extra bit for the 1 in normalized form. This module also checks if, after normalizing, the underflow and overflow condition is met or not. In the case of underflow, the output is 0 , and in the case of overflow, the output is infinity.
11. **Final_hardware** - This module checks for boundary conditions, i.e. if at least one of the input numbers is infinity, then it gives the output as infinity. This module takes fractional, exponent, and sign parts after all the calculations have been done and store them in a single 32-bit register, and gives it as the output.
12. **Float_adder** - This is the main module where all the input/output variables are defined, and the process of calling the different modules takes place.

Assumptions:

1. Here the inputs to the model are given in the IEEE 754 Format, and the inputs are always assumed to be in the normalized form.
2. The output of the model is also in IEEE 754 format and assumed to be in a normalized format.
3. The input numbers cannot be NaN. It should be a valid floating-point number or can be infinity as well.
4. Behavioural modelling has been followed, wherein the handshake concept has been incorporated by using the always block. The inputs to each module have been mentioned in the always block's sensitivity list, which means whenever input is ready/changes, the procedure mentioned in the always block is ready to be executed.
5. In the case of (+infinity – infinity), the output is considered to be + infinity.
6. If any one of the input is $\pm\infty$ then the output is $\pm\infty$.
7. No separate rounding hardware is added in the floating point adder. (only LSB bit is removed in case of carry).

Testbench:

There are 17 test inputs in the testbench, so approximately 1.8 micro seconds simulation time would be required to run all the test inputs.

test input 1 corresponds to time 100ns to 200ns

test input 2 corresponds to time 200ns to 300ns

and so on.

The Testcases are as follows:

The 1st number in comment is in decimal format and the 2nd number is in hexadecimal format.

```
//1 Subtracting a number from itself
num1=32'b1_10000001_000000000000000000000000; //-4 c0800000 // result = 0
num2=32'b0_10000001_000000000000000000000000; //+4 40800000
#100
//2 Num1-Num2 where |Num2|>|Num1|
num1=32'b0_10000001_000000000000000000000000; //+4 40800000 // result = -4 = c0800000
num2=32'b1_10000010_000000000000000000000000; //-8 c1000000
#100
//3 Num1-Num2 where |Num1|>|Num2|
num1=32'b0_10000001_000000000000000000000000; //+4 40800000 // result = 1.5 = 3fc00000
num2=32'b1_10000000_010000000000000000000000; //-2.5 c0200000
#100
//4 Num1-Num2 where |Num2|>|Num1|
num1=32'b0_10000000_010000000000000000000000; //+2.5 40200000 // result = -1.5 = bfc00000
num2=32'b1_10000001_000000000000000000000000; //-4 c0800000

//5 Num2-Num1 where |Num2|>|Num1|
num1=32'b1_10000000_010000000000000000000000; //-2.5 c0200000 // result = 1.5 = 3fc00000
num2=32'b0_10000001_000000000000000000000000; //+4 40800000
#100
//6 Overflow
num1=32'b0_11111110_111111111111111111111111; //7f7fffff // result = +infinity = 7f800000
num2=32'b0_11111110_100000000000000000000001; //7f400001
#100
//7 Underflow
num1=32'b1_11111110_111111111111111111111111; //ff7fffff // result = +infinity = ff800000
num2=32'b1_11111110_100000000000000000000001; //ff400001
#100
//8 normal floating point addition
num1=32'b0_10000001_110001000000000000000000; //7.0625 40e20000 // result = 8.09375 = 41018000
num2=32'b0_01111111_000010000000000000000000; //1.03125 3f840000

//9 +infinity + positive number
num1=32'b0_11111111_000000000000000000000000; // + infinity // output = +infinity
num2=32'b0_10000001_000000000000000000000000; //+4 40800000
#100
//10 +infinity + negative number
num1=32'b0_11111111_000000000000000000000000; // + infinity // output = +infinity
num2=32'b1_10000001_000000000000000000000000; //-4 c0800000
#100
//11 +infinity + 0
num1=32'b0_11111111_000000000000000000000000; // + infinity // output = +infinity
num2=32'b0_00000000_000000000000000000000000; // 0
#100
//12 -infinity + positive number
num1=32'b1_11111111_000000000000000000000000; // - infinity // output = -infinity
num2=32'b0_10000001_000000000000000000000000; //+4 40800000
```

```

//13 -infinity + negative number
num1=32'b1_11111111_000000000000000000000000; // - infinity // output = -infinity
num2=32'b1_10000001_000000000000000000000000; //-4 c0800000
#100
//14 -infinity + 0
num1=32'b1_11111111_000000000000000000000000; // - infinity // output = -infinity
num2=32'b0_00000000_000000000000000000000000; // 0
#100
//15 infinity + infinity
num1=32'b0_11111111_000000000000000000000000; // + infinity // output = +infinity
num2=32'b0_11111111_000000000000000000000000; // + infinity
#100
//16 infinity - infinity
num1=32'b0_11111111_000000000000000000000000; // + infinity // output = +infinity // by default
num2=32'b1_11111111_000000000000000000000000; // - infinity
#100
//17 -infinity - infinity
num1=32'b1_11111111_000000000000000000000000; // - infinity // output = -infinity
num2=32'b1_11111111_000000000000000000000000; // - infinity

```

Testbench Run:

All the outputs are mentioned in hexadecimal format.

Test Input no 1 to 4

Name	Value	100 ns	200 ns	300 ns	400 ns
▶ result[31:0]	ff800000	00000000	c0800000	3fc00000	bfc00000
▶ num1[31:0]	ff7fffff	c0800000	40800000	40200000	
▶ num2[31:0]	ff400001	40800000	c1000000	c0200000	c0800000

Test Input no 5 to 8

Name	Value	500 ns	600 ns	700 ns	800 ns
▶ result[31:0]	ff800000	3fc00000	7f800000	ff800000	41018000
▶ num1[31:0]	ff800000	c0200000	7f7fffff	ff7fffff	40e20000
▶ num2[31:0]	c0800000	40800000	7f400001	ff400001	3f840000

Test Input no 9 to 11

Name	Value	900 ns	1,000 ns	1,100 ns
▶ result[31:0]	ff800000		7f800000	
▶ num1[31:0]	ff800000		7f800000	
▶ num2[31:0]	ff800000	40800000	c0800000	00000000

Test Input no 12 to 14

Name	Value	1,200 ns	1,300 ns	1,400 ns
▶  result[31:0]	ff800000		ff800000	
▶  num1[31:0]	ff800000		ff800000	
▶  num2[31:0]	ff800000	40800000	c0800000	00000000

Test Input no 15 to 17

Name	Value	1,500 ns	1,600 ns	1,700 ns
▶  result[31:0]	ff800000	7f800000		ff800000
▶  num1[31:0]	ff800000	7f800000		ff800000
▶  num2[31:0]	ff800000	7f800000	ff800000	